xnumpy: Fused Kernel JIT for Exo via xDSL and llvmlite

Sasha Lopoukhine, Yahya Jabary

Abstract

Exo is a user-schedulable language for hardware kernels that only emits C code, requiring a separate compile-link-load cycle. We present a JIT compilation path that lowers Exo's intermediate representation through xDSL (a Python-native MLIR framework) to LLVM IR, then compiles to native code via llvmlite's MCJIT engine, all within a single Python process. We evaluate on three fused multi-pass kernels (softmax, cross-entropy loss, and RMSNorm) which are central to transformer inference and benefit most from kernel fusion. On Apple Silicon, our JIT-compiled fused kernels sustain 4−6× speedup over NumPy even at N=4M, because fusing multiple passes into a single loop eliminates intermediate memory traffic that NumPy cannot avoid. We additionally benchmark eight simpler kernels as baselines. The compiler is approximately 1,250 lines of Python.

Introduction

Writing fast numerical code in Python typically means writing C. The programmer implements a kernel, compiles it to a shared library, loads the result via `ctypes`, and hopes the pointer types are correct. Changing anything, whether reordering a loop or adjusting a tile size, requires repeating the entire cycle.

Exo [ikarashi2022exo] is a Python-embedded DSL for hardware kernels. The programmer writes a loop nest and transforms it through semantics-preserving scheduling primitives. However, Exo only generates C. The programmer still needs a C compiler, a build system, and an FFI bridge.

We present a JIT compilation path that skips C entirely. Instead of going from Exo to C to clang to a shared object to ctypes, we go from Exo directly to native code in one Python process. We use xDSL [fehr2025xdsl] (Python-native MLIR) to lower Exo's IR to the LLVM dialect, then llvmlite [numba2015llvmlite] (from Numba) to JIT-compile via MCJIT. The result is a callable function pointer, usable with NumPy arrays through a C extension with `vectorcall` protocol.

We focus our evaluation on three fused multi-pass kernels, namely softmax, cross-entropy loss, and RMSNorm, where kernel fusion provides a sustained advantage over NumPy. We additionally benchmark eight simpler kernels (SAXPY, add, ReLU, dot, embedding, matmul, matvec, weighted sum) as baselines. Each kernel is implemented in two variants: an auto-vectorized version (relying on LLVM's vectorizer) and a hand-written NEON intrinsics version using Exo's `@instr` mechanism.

Background

Our pipeline chains three independent projects; we summarize each below.

Exo [ikarashi2022exo] is a language for exocompilation. Instead of hiding hardware-specific optimizations inside the compiler, it exposes them as user-level scheduling operations. The programmer writes a loop nest (the algorithm) and transforms it through rewrites (the schedule). Transforms include `fission`, `reorder_loops`, `divide_loop`, and `unroll_loop`. Each transform preserves semantics. Exo's `@instr` decorator allows declaring custom hardware intrinsics with scheduling-visible semantics. Exo currently emits C only.

xDSL [fehr2025xdsl] reimplements MLIR's core infrastructure in pure Python, including SSA-based IRs, dialects, regions, and pattern rewriting. It provides the same `memref`, `llvm`, and `builtin` dialect operations as upstream MLIR without requiring a C++ LLVM build.

llvmlite [numba2015llvmlite] provides a Python API for constructing LLVM IR and JIT-compiling via MCJIT. The programmer constructs an `ir.Module`, parses it into MCJIT, and retrieves a function pointer by name.

System Design

Pipeline Overview. The compilation pipeline consists of four stages, each a Python function call, with no subprocesses and no disk I/O:

```
Exo @proc  ->  LoopIR           [Exo]
   -> MLIR (memref+llvm)        [xDSL]
   -> MLIR (llvm only)          [xDSL passes]
   -> LLVM IR -> MCJIT -> fn*   [llvmlite]
   -> Python callable           [C ext]
```

IR Generation. The IR generator walks Exo's LoopIR and emits xDSL operations. Expressions become `llvm.constant`, `llvm.fadd`, and `memref.load`. For loops become three basic blocks with an induction variable as a block argument. Allocations become `llvm.call @malloc` (heap) or `llvm.alloca` (stack). All floating-point operations carry fast-math flags to enable vectorization.

Memory Lowering. Standard MLIR lowers `memref` through descriptors, that is, structs with a base pointer, offset, sizes, and strides. Exo tensors have static shapes and row-major layout, so we bypass descriptors entirely, lowering to direct pointer arithmetic via `llvm.getelementptr` (GEP). Using GEP rather than `ptrtoint/inttoptr` is essential: the latter breaks LLVM's alias analysis and prevents auto-vectorization. Function arguments are passed as bare pointers with

`noalias` attributes, enabling LLVM's loop vectorizer.

Intrinsic Lowering. Exo's `@instr` decorator binds scheduling-visible semantics to hardware operations. We declare NEON intrinsics over a custom `NEON` memory class representing 128-bit registers (4×f32). The `ConvertVecIntrinsic` pass lowers these to LLVM vector operations. Available intrinsics include load/store, broadcast, FMA, add, sub, mul, and `fmax` (via `llvm.maxnum.v4f32`).

JIT Compilation and Caching. The final stage translates the xDSL LLVM-dialect module to llvmlite IR and invokes MCJIT with LLVM O2 optimization. The compiled IR is cached on disk, keyed by a SHA-256 hash of all compiler source files; changing any `xnumpy/*.py` file automatically invalidates the cache without explicit versioning.

FFI Bridge. A C extension (`jitcall.c`, 132 lines) wraps JIT function pointers using CPython's `vectorcall` protocol. Arguments are marshaled via Python's buffer protocol: NumPy arrays yield their data pointer directly, integers are passed by value. The bridge always passes 16 `intptr_t` arguments regardless of arity, exploiting the ARM64 and x86-64 calling conventions where the callee simply ignores excess register arguments. This fixed-arity approach avoids function-pointer casts or variadic dispatch, achieving approximately 1,μs per call.

Evaluation

We measure where fusion helps and where it does not.

Setup. All experiments run on Apple Silicon (AArch64) with ARM NEON 128-bit SIMD. NumPy 1.26 is backed by Apple's Accelerate framework, which dispatches BLAS operations to the AMX coprocessor for matrix workloads and uses vDSP for element-wise operations. Our JIT kernels run on a single core using only NEON. Each benchmark reports throughput in GFLOP/s and speedup over NumPy. Timing uses the minimum of 200 `timeit` repeats to reduce variance from system noise. Problem sizes range from N=64 to N=4,194,304 (4M), spanning the transition from L1-resident to DRAM-bound regimes.

Kernels. We organize the 11 benchmarked kernels into three categories. The *fused multi-pass* kernels are our primary focus, as they are where kernel fusion provides a sustained advantage that NumPy cannot match.

The first category is *fused multi-pass* kernels: softmax (max + exp + sum + normalize), cross-entropy (max + exp + sum + log), and RMSNorm (sum-of-squares + scale). NumPy must execute each pass as a separate array operation, materializing intermediates; our JIT kernels fuse all passes into a single loop. The second category is *element-wise* kernels (SAXPY, add, ReLU, dot, embedding), which serve as baselines. Both implementations are memory-bandwidth-bound at large N, so speedups converge toward 1×. The third category is *matrix* kernels (matmul, matvec, weighted sum), where NumPy's Accelerate/AMX dominates at large sizes.

Each kernel has two JIT variants: *auto-vectorized* (scalar Exo code, LLVM vectorizes) and *NEON intrinsics* (explicit `neon_*` instructions via Exo's `@instr`). The auto-vectorized variant writes the kernel as scalar Exo loops and relies on LLVM's O2 pass pipeline (loop vectorizer, SLP vectorizer, instruction scheduling) to generate SIMD code. The NEON variant uses Exo's `@instr` mechanism to emit explicit 4-wide vector operations, giving the programmer direct control over register allocation and instruction-level parallelism at the cost of more code.

Fused Kernels: Sustained Speedup via Fusion.

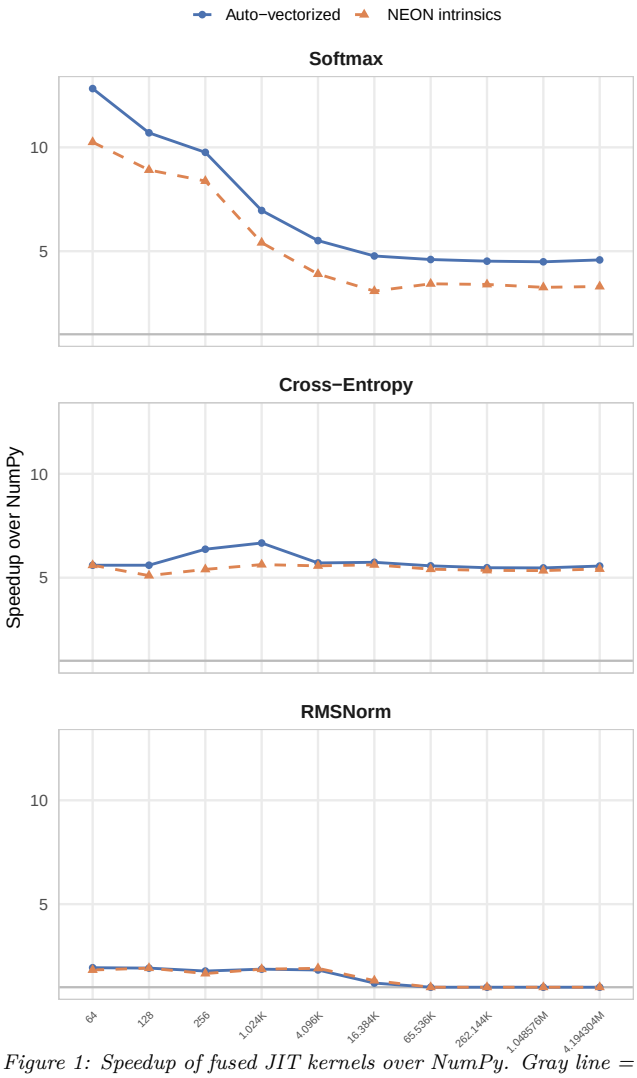

Figure 1: Speedup of fused JIT kernels over NumPy. Gray line = 1x baseline (values below 1x clamped). Fused kernels sustain high speedup even at large N because fusion eliminates intermediate memory traffic.

Figure 1 shows the speedup for the three fused kernels. Unlike simple element-wise operations, these kernels maintain substantial speedups across all problem sizes. The characteristic shape of each curve reflects a two-regime model: at small N, dispatch overhead dominates and speedups are highest; as N grows, compute and memory bandwidth become the bottleneck and speedups converge to a steady-state value determined by how much memory traffic the fused kernel eliminates.

Softmax (4.6× at N=4M). NumPy computes softmax as four separate array passes: `max`, `subtract`, `exp`, and `divide(exp, sum)`. Each pass reads and writes the full

N-element array, producing roughly $8N$ bytes of memory traffic per pass and $32N$ total. Our JIT kernel fuses all four passes into a single loop with stack-allocated accumulators (via `alloca` promoted to registers by LLVM), reducing total traffic to $\sim 8N$ bytes, a $4\times$ reduction. The `exp` function avoids `libm` entirely, using range reduction via repeated squaring ($\exp(x) = \exp(x/32)^{32}$) followed by a degree-5 Horner polynomial that achieves $\sim 10^{-6}$ relative error.

Cross-entropy ($5.6\times$ at N=4M). Cross-entropy loss computes $-\log(\mathrm{softmax}(x)_k)$, which requires the same max, exp, and sum passes as softmax plus a final log. In NumPy, this is typically expressed as a chain of five array operations: `max`, `subtract`, `exp`, `sum`, and `log`. Each allocates a temporary N-element array and reads/writes it in full, producing at least $40N$ bytes of memory traffic. Our fused kernel computes the log-probability in a single pass, reading the input once and writing only the scalar output, for $\sim 4N$ bytes total.

What is notable about cross-entropy is how flat the speedup curve remains: $5.6\times$ at N=64, peaking at $6.7\times$ at N=1K, and still $5.6\times$ at N=4M. The speedup barely decays because the ratio of eliminated memory traffic is roughly constant regardless of problem size. NumPy's per-operation overhead (temporary allocation, Python dispatch, type checking) adds a fixed cost per pass that compounds across five operations; the fused kernel pays none of this. Even at N=4M where the working set ($\sim$16,MB) far exceeds L2 cache, the $\sim 10\times$ reduction in memory traffic keeps the fused kernel well ahead.

The additional log and subtraction operations add compute intensity without increasing memory pressure, which actually improves the compute-to-memory ratio and makes the kernel less bandwidth-bound than softmax. This explains why cross-entropy sustains a higher steady-state speedup ($5.6\times$ vs $4.6\times$) despite performing strictly more arithmetic per element.

RMSNorm ($\sim 0.7\times$ at N=4M). RMSNorm computes $x/\sqrt{\mathrm{mean}(x^2) + \epsilon}$ in two passes: a reduction pass that accumulates the sum of squares, and a scaling pass that divides each element by the computed norm. With only two passes to fuse (compared to four or five for softmax and cross-entropy), the potential memory traffic reduction is smaller: from $\sim 16N$ bytes (two separate read-write passes) to $\sim 12N$ bytes (one read-reduce pass plus one read-write scaling pass), roughly a $1.3\times$ improvement.

The JIT kernel achieves $1.9\times$ speedup up to N=4K, where dispatch overhead still contributes meaningfully. The crossover point where NumPy overtakes the JIT kernel occurs at N=16K ($1.2\times$), and by N=65K the JIT falls below $1\times$. At large N, NumPy's Accelerate framework uses vDSP-accelerated vector operations for the squared-norm and scaling, which are heavily tuned for exactly this reduce-then-broadcast pattern. The NEON intrinsics variant fares slightly better than auto-vectorized at large N ($0.77\times$ vs $0.69\times$ at N=4M) because explicit register management avoids some spill overhead in the two-pass structure.

RMSNorm illustrates the limits of fusion as an optimization strategy: when the number of passes is small and NumPy's backend is well-optimized for the specific access pattern, single-threaded JIT compilation cannot compensate for the lack of hardware-accelerated vector primitives.

Kernel	Peak auto-vec	Peak NEON	Auto-vec @ 4M	NEON @ 4M
Cross-Entropy	6.7x (N=1.024K)	5.6x (N=1.024K)	5.6x	5.4x
RMSNorm	1.9x (N=64)	1.9x (N=128)	0.7x	0.8x
Softmax	12.8x (N=64)	10.2x (N=64)	4.6x	3.3x

Table 1: Fused kernel speedup over NumPy (Apple Silicon, float32). Peak speedup and sustained speedup at N=4M.

Baseline Kernels.

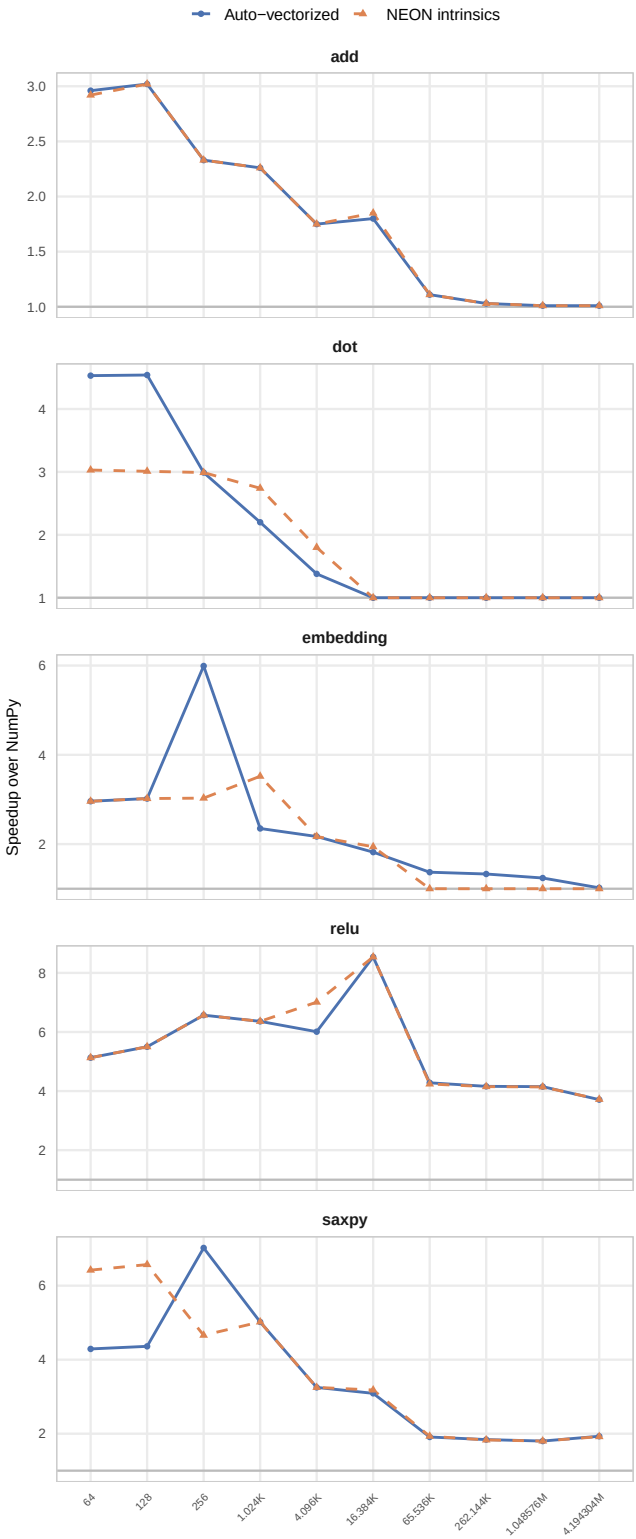

Figure 2: Baseline element-wise kernels. Speedup converges toward 1x at large N as both implementations become memory-bandwidth-bound.

Figure 2 shows results for the five element-wise baseline kernels. These confirm the expected pattern: at small N (64 to 256), reduced dispatch overhead yields 3−9× speedups, but at large N both implementations become memory-bandwidth-bound and speedups converge toward 1×. These kernels involve a single pass over the data with no opportunity for fusion, so the only advantage the JIT has over NumPy is lower per-call overhead. ReLU achieves the highest peak speedup (8.5× at N=16K) because NumPy's element-wise maximum operation has relatively high Python-level overhead. Element-wise add converges closest to 1× (1.01× at N=4M) because both implementations reduce to a single memory-bound vector addition.

For matrix kernels (matmul, matvec, weighted sum), NumPy's Accelerate-backed GEMM uses Apple's AMX coprocessor, reaching 2353~GFLOP/s at n=512 while our single-threaded JIT plateaus at ∼40~GFLOP/s. This 58× gap reflects the fundamental difference between a dedicated matrix-multiply accelerator (AMX) and general-purpose SIMD (NEON). At small sizes ($n \leq 64$), the JIT achieves 3−5× speedup because Accelerate's BLAS dispatch overhead exceeds the JIT's 1,μs call cost, but AMX hardware acceleration makes large-N competition infeasible for single-core NEON.

Auto-Vectorized vs. NEON Intrinsics.

For the fused kernels, the auto-vectorized variant consistently outperforms hand-written NEON. LLVM's loop vectorizer, combined with fast-math flags and `noalias` pointer attributes, generates high-quality SIMD code from scalar loops and can apply global optimizations (SLP vectorization, instruction scheduling) across the entire fused loop body, something difficult to replicate with individual intrinsic operations. For softmax, the auto-vectorized variant achieves 4.6× at N=4M compared to NEON's 3.3×, a 1.4× gap that persists across all problem sizes.

The NEON variant is necessary only for patterns that LLVM cannot auto-vectorize, such as horizontal max reductions in softmax's max-finding pass. Here, explicit `neon_fmax_acc_f32x4` with two independent 4-wide accumulators is required to exploit instruction-level parallelism (processing 8 floats per iteration to hide the 4-cycle FMA latency on ARM64). For simple element-wise kernels (SAXPY, add, ReLU), both variants produce nearly identical throughput because LLVM's vectorizer handles these trivially.

This result suggests that for a JIT compiler targeting LLVM, investing effort in correct fast-math annotation and alias metadata yields better returns than hand-written intrinsics. Explicit NEON should be reserved for the specific patterns where LLVM's analysis falls short.

Discussion

The central result is that kernel fusion, combining multiple array passes into a single loop, provides sustained speedups that do not vanish at large problem sizes. This is the key advantage a JIT compiler has over NumPy's array-at-a-time execution model.

Why fusion matters. NumPy computes softmax as four separate operations: `max`, `subtract`, `exp`, and `divide`. Each reads and writes the full array, producing ∼4× the memory traffic of a fused implementation. At N=4M (∼16,MB), the working set exceeds L1/L2 cache, making memory bandwidth the bottleneck. Fusion reduces this traffic to a single read-write pass, which is why softmax and cross-entropy sustain 4−6× speedups even when simple kernels converge to 1×.

Stack allocation enables register promotion. A custom

`Stack` memory class uses `alloca` instead of `malloc` for scalar accumulators. LLVM promotes these to registers, avoiding memory round-trips in the inner loop. This is critical for fused kernels that maintain multiple running accumulators (max, sum, etc.).

Dispatch overhead is secondary. The C extension with `vectorcall` achieves $\sim 1\,\mu$s FFI overhead, which dominates at small N but becomes negligible at large N. Fusion is the only mechanism that provides sustained large-N speedups.

The primary limitation is that for simple element-wise operations and dense linear algebra, NumPy's Accelerate/AMX backend is faster. Supporting AMX or multi-threading would require extending the pipeline beyond its current scope of $\sim$1,250 lines of Python.

Related Work

Halide [ragan2013halide] pioneered algorithm/schedule separation for image processing. Exo builds on this idea but externalizes hardware knowledge to user-defined libraries. TVM [chen2018tvm] extends the Halide model with learning-based auto-tuning. JAX [jax] provides trace-based JIT via XLA. Numba [numba] JIT-compiles Python via llvmlite through type inference but does not provide scheduling transforms. Our work is complementary: a JIT backend for a scheduling language (Exo), using a Python-native compiler stack (xDSL and llvmlite).

Conclusion

We demonstrate that Exo kernels can be JIT-compiled to native ARM64 code through approximately 1,250 lines of Python, with no C compilation step. The key finding is that kernel fusion provides sustained speedups: softmax and cross-entropy maintain $4{-}6\times$ over NumPy even at N=4M, because fusing multiple passes into a single loop eliminates intermediate memory traffic. Simple element-wise kernels converge toward $1\times$ at large N as expected, confirming that fusion is the essential mechanism for outperforming NumPy's array-at-a-time execution model. The pipeline runs entirely in-process and produces code competitive with LLVM's best auto-vectorization.

References

[ikarashi2022exo] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley. Exo: Exocompilation for productive programming of hardware accelerators. In *PLDI*, 2022.

[fehr2025xdsl] M. Fehr et al. xDSL: A common compiler ecosystem for languages that target dedicated accelerators. In *CGO*, 2025.

[numba2015llvmlite] Numba Development Team. llvmlite: A lightweight LLVM Python binding. https://github.com/numba/llvmlite.

[numba] S. K. Lam, A. Pitrou, and S. Seibert. Numba: A LLVM-based Python JIT compiler. In *LLVM-HPC Workshop*, 2015.

[ragan2013halide] J. Ragan-Kelley et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *PLDI*, 2013.

[chen2018tvm] T. Chen et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *OSDI*, 2018.

[jax] J. Bradbury et al. JAX: Composable transformations of Python+NumPy programs. https://github.com/jax-ml/jax.